Uploading a File to SFTPGo Using Python

Overview

This guide provides step-by-step instructions on how to connect to an **SFTPGo** server and upload a file using Python's paramiko library.

Prerequisites

Before running the script, ensure the following:

- You have **Python 3.x** installed.
- You have installed the paramiko library. If not, install it using:
- pip install paramiko
- You have valid **SFTPGo credentials** (hostname, username, password).
- You have the **local file** ready for upload and know the **remote directory path**.

Python Script to Upload a File to SFTPGo

```
import paramiko


# SFTPGo server details
hostname = "your-sftpgo-server.com"  # Replace with your SFTPGo server hostname or IP
port = 22  # Default SFTP port (change if necessary)
username = "your_username"  # Replace with your SFTPGo username
password = "your_password"  # Replace with your SFTPGo password


# Local file path (file to be uploaded)
local_file = "path/to/local/file.txt"  # Replace with your local file path


# Remote file path (destination on SFTP server)
remote_path = "/file.txt"  # Replace with the remote directory path


try:
    # Step 1: Establish Connection
    transport = paramiko.Transport((hostname, port))
    transport.connect(username=username, password=password)
```

```
    # Step 2: Open SFTP session
    sftp = paramiko.SFTPClient.from_transport(transport)
    print("✅ Connected to SFTPGo successfully!")

    # Step 3: Upload the file
    sftp.put(local_file, remote_path)
    print(f"📤 File '{local_file}' uploaded successfully to '{remote_path}'.")

    # Step 4: Close the connection
    sftp.close()
    transport.close()
    print("🔒 Connection closed.")

except Exception as e:
    print(f"❌ Error: {e}")
```

Steps Explained

1. Connect to SFTPGo Server

- Establishes an **SSH transport connection** using paramiko.Transport.
- Authenticates using a **username and password**.

2. Open an SFTP Session

- Creates an **SFTP client** using paramiko.SFTPClient.from_transport().

3. Upload the File

- Uses sftp.put(local_file, remote_path) to transfer the file.
- Ensure the remote_path exists; otherwise, an error may occur.

4. Close the Connection

- The script **closes the SFTP session and transport connection** to free resources.

Additional Features

Verify File Upload

After uploading, confirm the file exists:

```
if remote_path in sftp.listdir("/"):
    print("✅ File uploaded successfully!")
else:
    print("❌ Upload failed!")
```

Conclusion

This script provides a simple way to **connect, upload, and verify file transfers** to an SFTPGo server using Python.